

Hands on 4

Difference between JPA, Hibernate and Spring Data JPA

Objective

To understand the differences between JPA, Hibernate, and Spring Data JPA and learn how they work together in Java applications.

1. Java Persistence API (JPA)

JPA (Java Persistence API) is a Java specification (JSR 338) that defines standards for storing, retrieving, and managing data between Java objects and relational databases.

Features

- Specification, not an implementation
- Provides annotations like `@Entity`, `@Table`, `@Id`, and `@Column`
- Defines standard APIs for ORM
- Requires an implementation such as Hibernate

2. Hibernate

Hibernate is an Object Relational Mapping (ORM) framework that implements the JPA specification.

Features

- Maps Java objects to database tables
- Generates SQL queries automatically
- Supports CRUD operations
- Handles relationships between entities
- Provides caching and transaction management

3. Spring Data JPA

Spring Data JPA is a Spring Framework module built on top of JPA.

Features

- Reduces boilerplate code
- Provides `JpaRepository`
- Generates CRUD methods automatically
- Uses Hibernate internally

- Supports automatic transaction management

Relationship

Spring Data JPA



Hibernate



JPA Specification



MySQL Database

Comparison Table

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	Framework	Spring Module
Implementation	No	Yes	Uses Hibernate
ORM Support	Defines Rules	Yes	Yes
CRUD Operations	API only	Manual	Automatic
Transaction Management	No	Manual	Automatic
Repository Support	No	No	Yes (JpaRepository)
SQL Generation	No	Yes	Uses Hibernate

Hibernate Example

```
Session session = factory.openSession();
```

```
Transaction tx = session.beginTransaction();
```

```
session.save(employee);
```

```
tx.commit();
```

```
session.close();
```

Observation

The developer needs to manage sessions and transactions manually.

Spring Data JPA Example

```
@Repository
```

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
```

```
}
```

```
@Service
```

```
public class EmployeeService {
```

```
    @Autowired
```

```
    private EmployeeRepository employeeRepository;
```

```
    @Transactional
```

```
    public void addEmployee(Employee employee) {
```

```
        employeeRepository.save(employee);
```

```
    }
```

```
}
```

Observation

Spring Data JPA automatically manages sessions and transactions, reducing boilerplate code.

Advantages of Spring Data JPA

- Less coding effort
- Automatic CRUD implementation
- Better readability
- Easier maintenance

- Automatic transaction management
- Seamless integration with Spring Boot

Conclusion

JPA is a specification that defines persistence standards. Hibernate is an implementation of JPA that performs Object Relational Mapping. Spring Data JPA is a higher-level abstraction that simplifies data access by reducing boilerplate code and providing repository interfaces for database operations

Learning Outcome

After completing this exercise, I understood:

- The difference between JPA, Hibernate, and Spring Data JPA.
- How Hibernate implements JPA.
- How Spring Data JPA simplifies database operations.
- Why Spring Data JPA is preferred in modern Spring Boot applications.